





Structure des programmes

Sommaire

- 1. Structure de l'environnement Synaxis**
- 2. Organisation des modules/programmes**
- 3. Variables globales vs locale**
- 4. Charte syntaxique et indentation**
- 5. Initialisation du robot**
- 6. Apprentissage des points**
- 7. Documentation**

1. Structure de l'environnement Synaxis

Menu de la cellule Menu de l'éditeur

Raccourcis éditeur

Vue des tâches en cours

Structure des modules

Variables globales

Type de variables globales

Entrées de la fonction/proc.

Variables locales

Zone d'édition du code de la fonction/procédure

Console

Retour de la fonction

Fenêtre floating

Zone de simulation

Raccourcis de la cellule

Informations d'état du workspace/application

Hes-so
Haute École Spécialisée
du Sud de l'Occidentale

The screenshot displays the Synaxis software environment. At the top, there are two menus: 'Menu de la cellule' (Cell Menu) and 'Menu de l'éditeur' (Editor Menu). Below these are toolbars for 'Raccourcis éditeur' (Editor Shortcuts) and 'Raccourcis de la cellule' (Cell Shortcuts). The main workspace is divided into several sections: a left sidebar for 'Vue des tâches en cours' (Current Task View) and 'Structure des modules' (Module Structure); a central area for 'Entrées de la fonction/proc.' (Function/Procedure Inputs) and 'Variables locales' (Local Variables); a large 'Zone d'édition du code de la fonction/procédure' (Function/Procedure Code Editing Zone) with a code editor; a 'Console' at the bottom; and a 3D 'Zone de simulation' (Simulation Zone) on the right showing a robotic arm. A 'Fenêtre floating' (Floating Window) is also visible. Annotations include 'Type de variables globales' (Global Variable Type) pointing to a dropdown, 'Retour de la fonction' (Function Return) pointing to a button, and 'Raccourcis de la cellule' pointing to the right sidebar. The bottom status bar shows 'Informations d'état du workspace/application' (Workspace/Application Status Information) and the 'Hes-so' logo.

2. Organisation des modules

Votre workspace est organisé en modules.

Ces modules sont simplement un ensemble de procédures/fonctions.

On peut utiliser un module pour regrouper des procédures/fonctions utiles (sorte de librairie) ou des procédures/fonctions liées à la même application.

Dans Synapxis, tous les bouts de codes créés sont appelés «programmes».

Dans le cours, nous ferons la différence entre une procédure (ne retourne pas de valeurs, modifie l'état courant de l'application) et une fonction (retourne une valeur, ne modifie pas l'état courant)

3. Variables globales vs locales

Il existe deux zones de déclaration des variables dans Synaxis.

Vous avez la possibilité de déclarer des **variables locales**, utilisées uniquement au sein d'un programme ou fonction, ou des **variables globales**, atteignables par n'importe quel programme/module.

Les variables globales peuvent encore être séparée dans deux catégories:

Temporaire: Pas de valeur par défaut, **réinitialisées à chaque démarrage** de Synaxis

Persistantes: A chaque démarrage, la variable garde la dernière valeur **que l'on lui a attribuée (Attention Sauvegarde nécessaire!)**

4. Charte syntaxique

Si une charte syntaxique a été enseignée au cours de programmation de l'année précédente, c'est elle qui doit être appliquée lors de la programmation des exercices. Si aucune règle n'est en vigueur, il est demandé que les règles suivantes soient respectées pour la création des identifiants, notamment les variables globales, locales et les programmes :

Seuls les caractères **a-z**, **A-Z**, **0-9** et **" _ "** peuvent être utilisés.

Un identifiant ne commence jamais par un chiffre.

Il est conseillé de séparer les différentes parties du nom par des majuscules.

Le nombre de caractères n'est pas limité, mais le mot doit rester lisible.

Le programme doit être **indenté** c'est-à-dire que le code se trouvant dans chaque boucle doit être décalé par une tabulation.

4. Charte syntaxique - Indentation

Exemple :

```
If(i==0)  
->  Ex_pickOrPlace(true)  
else  
->  Ex_pickOrPlace(false)  
end
```

```
for(i=0 ; i<=n ; i+=1)  
->  for(j=0 ; j<=n ; j+=1)  
->->  Ex_pickOrPlace(i, j)  
->  end  
end
```


4. Charte syntaxique – Indentifiants

Le nom d'une **variable globale** commence par la lettre minuscule "g" : **gToolName, gCycleDuration, ...**

Dans le cas d'une position cartésienne, ou articulaire, la deuxième lettre représente le type, en minuscule : **gjPosStart, gcFrameOri, gcFrameX, gcFrameY, ...**

Le nom d'une variable locale commence par une lettre minuscule: **paletDx, paletDy,...**

Certaines variables locales peuvent ne comporter qu'une seule lettre : **i, n, ...**

Le nom d'un programme commence par une majuscule : **ExInit(), ExStart(), ExEnd()**

Le nom d'un module commence par une majuscule : **Ex_01, AppUtilities, ...**

5. Initialisation du robot

Pour chacun des exercices, les étapes d'initialisation fondamentales doivent impérativement être exécutées :

- Sélection du robot utilisé pour la tâche courante :
deviceAttach(gRobotName)
- Sélection du MCP utilisé pour le dialogue avec l'utilisateur :
deviceAttach(gMcpName)
- Sélection du mode de marche et mise en puissance du bras :
ensure(true)
- Initialisation du tool (transformée du préhenseur) :
setTool(machineTool(gToolName))
- Initialisation de la vitesse initiale (faible) :
setSpeed(gSpeedSlow)

5. Initialisation du robot

Une application robotique comporte généralement plusieurs préhenseurs différents. De la même manière, la réalisation de ces exercices de base nécessite l'utilisation d'au moins 2 préhenseurs, avec 2 transformées géométriques différentes (**tool**) :

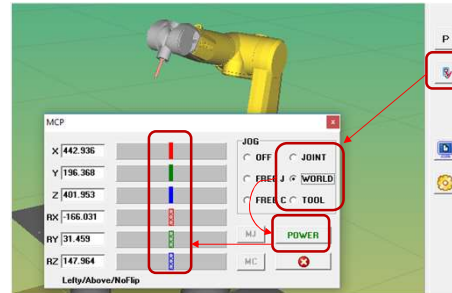
Pointe d'apprentissage / Ventouse vacuum

Il est conseillé d'utiliser une variable globale temporaire qui désigne le nom du préhenseur qui doit être utilisé lors de l'initialisation (**gToolName**).

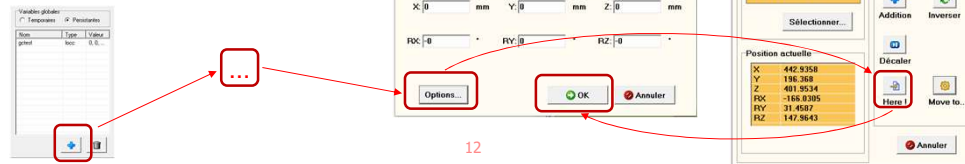
Lors du chargement de l'application, cette variable globale a pour valeur par défaut une string vide (""). Ainsi, lors de la première exécution, il est possible de détecter qu'elle n'a jamais été initialisée, et d'interroger l'utilisateur pour choisir quel préhenseur doit être utilisé.

6. Apprentissage des points

Pour la réalisation des exercices, vous aurez régulièrement besoins d'apprendre des points avec le robot. Cette action est possible également avec le simulateur:



- Se déplacer grâce au MCP virtuel
- Définir une variable globale **gc/j...**
- Avec **Option** -> **Here**, lui affecter la position courante



7. Documentation

Vous trouvez les documentations concernant Synpaxis dans le répertoire d'installation du programme. En principe:

«C:\Program Files (x86)\Synapaxis\Documentation»

Dans ce répertoire, vous trouver 3 documents:

Synapaxis_Doc_General: Documentation générale de Synpaxis

Synapaxis_Doc_Lang: Définit le langage utilisé dans Synpaxis

Synapaxis_Doc_Trajectoire: Gestion des trajctoirs complexes, pas utilisées dans ce cours.

Les deux premiers sont également présents dans le répertoire du cours.